PRISM Experiments Audit Report

Date: 2026-07-12 · **Repository:** github.com/bleymambwe/PRISM (main)

Purpose: a self-contained, auditable record of every experiment and benchmark run in the PRISM program: the problem each one addressed, exactly what was done and how, the configurations (verbatim where it matters), the results, what they mean, and what comes next — including the tool-calling / agentic direction. Formats: this Markdown file and `EXPERIMENTS_AUDIT_REPORT_2026-07-12.pdf` built from it.

1. How to audit this document

Every number below is copied from committed artifacts, never from memory. For any claim:

- The **raw data** is a CSV in the experiment folder given at the end of each section (e.g. `experiments/iteration-09/results/llm_landscape.csv`). API experiments keep a per-(ordering, question) answer cache — the fitness of any ordering can be recomputed by averaging its 32 (or 100) cached 0/1 rows.
- The **code that produced it** is in the same folder; all runs are seeded and, for API work, cached and resumable, so re-running a script reproduces (or resumes) the CSV.
- The **narrative record** is `docs/EXPERIMENT_LOG.md` (newest-first) plus one write-up per iteration in `iterations/`. Decisions are in `docs/DECISION_LOG.md` (D8–D21).
- **Pre-registration:** pre-flight forecasts were committed to the log *before* the corresponding searches ran; the git history timestamps prove the ordering.
- Statistical bar (D18): any headline requires ≥40-seed confidence intervals or exact enumeration. Where an early claim failed this bar it was retracted, and the retraction is part of this report (§6).

Total program cost: **under $20** across 19 iterations. No GPU was ever used; everything ran on a Windows 10 laptop (CPU-only torch) using kill-safe chunked runners.

2. One clarification up front: permutations only, or the full genetic cycle?

Both, and the distinction matters for auditing — each experiment below states which mode produced its numbers.

The PRISM searcher is a full evolutionary loop, with one deliberate omission. When the searcher runs, it is a real population-based genetic algorithm:

- **Population** (typically 20; 40 at n≥7 per D12) of candidate permutations;
- **Selection:** tournament selection, k = 3;
- **Variation:** *mutation only* — swap, insert, inversion, or scramble moves on the permutation, or the uniform **portfolio** (a random operator per mutation event, D8); mutation probability 0.05 historically, 0.5–0.6 at n≥7 (D12);
- **Replacement:** elitism (best-ever never lost — the original convergence theorem) or **aging** (oldest dies, per regularized evolution — record-convergence theorem, Appendix B), or the elitist hybrid;
- **No crossover.** PRISM has never used recombination or EDA-style distribution sampling. This is a scoped design choice (permutation crossover operators like OX/PMX are a listed future extension, opportunity R7), so PRISM is precisely a *mutation-based evolutionary algorithm over the symmetric group S_n*.

But many headline numbers involve no genetic operations at all. The landscape results — the 720-ordering LLM map, the SciML pipeline map, all v4 neural benchmarks, the transfer studies — come from **exhaustive enumeration or fixed random sampling** of orderings, evaluated deterministically or via cached API calls. The searcher was then raced *against* those cached landscapes (so search comparisons are exact and free). And per the protocol-vs-searcher distinction (D21), the *protocol* may select enumeration, random sampling, a surrogate model, or the EA — the EA is one executor.

Mode	Experiments that used it
Exhaustive enumeration (no genetic ops)	G, H-StageA, I (landscape), K, M-gate, P, Q, Q2, O, R
Full EA (selection+mutation+elitism/aging)	A, C/D, E, F, G/H/I (search stage), K/M (search stage), L, N (as baseline)
Surrogate model (ridge regression, no EA)	N (winner), L (BANANAS-style)
Fixed random sampling (baseline or statistics)	every search comparison (mandatory per D12)

3. Global experimental configuration

Item	Value
Machine	Windows 10 Home, CPU-only (no GPU ever), Python 3.12
Run discipline	background processes die in ~3–8 min → every long run appends one CSV row per completed unit, resumes by skipping recorded keys, takes a wall-clock budget argument, and runs as repeated foreground chunks
Seeds	fixed and recorded everywhere (e.g. seed 42 reproduction; seed 17 ordering samples; seed 99/42 bootstrap)
LLM decoding	temperature 0, capped output tokens, exact-match grading with a frozen regex parser
Cost control	hard call caps in code; It-19 added a two-meter $9 guard (per-call API-reported cost in `spend_guard.json` + independent key-usage endpoint poll)
Statistics	Wilson/bootstrap CIs; 40 seeds for headlines (D18); 10k bootstrap resamples for transfer CIs
Secrets	`GOOGLE_API_KEY`, `OPENROUTER_API_KEY`, `OPENAI_API_KEY` in GCP Secret Manager only; `.env` files gitignored and deleted

4. The experiments, one by one

Each entry: **Problem** → **Setup/config** → **What was done** → **Results** → **Meaning** → **Artifacts.**

4.1 Experiment A — Reproduction of the historical toy results (Iteration 2)

- **Problem:** the original handover described results (XOR/OR/AND/3-bit-parity classification and a polynomial regression solved by permuting 5 neural branch types) from a codebase that turned out not to exist. Can the claims be reproduced at all?
- **Setup:** rebuilt implementation; population 20, tournament k=3, swap mutation p_m=0.05, elitism 1, 150 generations, seed 42; numpy 1.26.4, torch 2.12.1+cpu; ~34 min.
- **Mode:** full EA.
- **Results:** all headline numbers reproduce — accuracy 1.0000 on the four classification tasks; polynomial MSE 0.0070 (reference 0.0068). Three corrections: AND has *zero* ordering variance (order matters for 4/5 tasks only); "convergence generation" is meaningless under stochastic fitness (gen-0 lucky hits); only permutation positions 0–2 are functional.
- **Meaning:** claims survive, benchmarks were weaker than advertised → benchmark redesign.
- **Artifacts:** `prism-research/experiments/validate_toy_problems.py`, `prism-research/outputs/validation_*`, tag `iteration-02`.

4.2 Experiments C & D — Operator × landscape study (Iteration 3)

- **Problem:** does the mutation operator have to *match* the landscape type (Cicirello's operator–landscape theory), and what happens on a deceptive landscape?

- **Setup:** operators {swap, insert, inversion, scramble} × synthetic landscapes {hamming = absolute-position, kendall = precedence, adjacency = neighbor/routing} × n ∈ {6,8,10,12,14,16}, 15 seeds, cap 10,000 generations; deceptive landscape {swap, inversion} × n ∈ {6,8,10}. Kill-safe incremental CSV with resume.
- **Mode:** full EA (this is a pure searcher study).
- **Results:** matched operators best on 3/3 landscape types — swap/hamming 405 generations at n=16 vs **total censoring** (never finishing) for all others; insert/kendall best at 5/6 sizes; inversion/adjacency 1,199 at n=16 vs total censoring. Smooth landscapes forgive mismatch (10–40% penalty); plateau-rich landscapes turn mismatch into 100% failure. Matched scaling exponents 2.79 / 2.98 / 3.44 (consistent with $O(n^3 \log n)$ to n=16). Deceptive landscape: censors nearly everything by n=8–10.
- **Meaning:** operator choice is the difference between polynomial search and guaranteed failure — and it depends on a measurable landscape property.
- **Artifacts:** `experiments/iteration-03/operator_study.py`, `results/operator_*`.

4.3 Experiment E — All-positions benchmark attempt (Iteration 3) — NEGATIVE

- **Problem:** make all permutation positions functional by deepening the networks.
- **Result:** falsified — variance across orderings dropped to exactly 0 (deeper stacks untrainable in budget; every ordering at chance 0.500), while the search loop still reported best fitness 1.0 — proving best-ever fitness under noise inflates.
- **Meaning:** never trust best-ever fitness without ground truth → deterministic benchmarks became mandatory.
- **Artifacts:** `experiments/iteration-03/validate_toy_v3.py`, `results/toy_v3_results.csv`.

4.4 Experiment F — Operator portfolio (Iteration 4)

- **Problem:** can search succeed *without* knowing the landscape type?
- **Setup:** uniform portfolio (random operator per mutation event) and an adaptive credit-assignment variant; same protocol as C for direct merging.
- **Mode:** full EA.
- **Results:** portfolio = zero censoring on every landscape at 1.1–4.1× matched-operator cost (vs unbounded failure for mismatched fixed operators). Adaptive variant correctly ranks the matched operator top 3/3 (an automated landscape typer) but isn't faster.
- **Meaning (D8):** portfolio = default under ignorance; matched operator when the type is known.
- **Artifacts:** `experiments/iteration-04/`.

4.5 Experiment G — Benchmark v4, exact ground truth (Iteration 5)

- **Problem:** build toy neural benchmarks where every ordering's fitness is a deterministic, exactly enumerable number.
- **Setup:** residual blocks (trainability) + permutation-seeded initialization (determinism); all 120 orderings enumerated per task (k=3 seeded trials each); searcher (portfolio, 60 generations, 10 seeds) raced against the enumerated cache.
- **Mode:** enumeration (landscape) + full EA (race).
- **Results:** XOR-v4 optimum 1.0000 held by 8/120, std 0.120, PRISM hit rate 100%. Parity-v4 optimum 0.9583 held by 2/120, std 0.098, hit rate 40%, mean regret 0.0333. Determinism verified by repeat evaluation.
- **Meaning (D9):** canonical benchmark design — search quality as exact hit rate and regret, not noisy best-ever.
- **Artifacts:** `prism-research/benchmarks/toy_problems_v4.py`, `experiments/iteration-05/`.

4.6 Experiment H — Scale-up (Iteration 6)

- **Setup/results:** Stage A n=6 XOR: 720 enumerated, optimum 33/720, hit rate 0.90, regret 0.0083 — scales. Stage B n=7 XOR k=1: PRISM and random both saturate at 1.0 — optimum set too dense to discriminate. (A chunk-overlap incident wrote 103 duplicate rows; all values identical — an accidental determinism check that passed.)
- **Meaning:** discrimination needs sparse optima + evaluations-to-first-optimum → Exp. I.

- **Artifacts:** `experiments/iteration-06/`.

4.7 Experiment I — n=7 parity race (Iteration 7) — H10 FALSIFIED (pivotal)

- **Problem:** with sparse optima and a fair cost model, does the searcher actually beat random sampling on a neural landscape?
- **Setup:** all 5,040 orderings of parity n=7 enumerated (optimum held by 14/5,040 = 0.28%; std 0.113); race under distinct-evaluation cost; 15 seeds.
- **Mode:** enumeration (landscape) + full EA vs random (race).
- **Results:** default EA (pop 20, p_m 0.05): 4/15 hits — premature convergence, ~103 distinct orderings explored. Random: 100% hits, mean 318 evals. Tuned EA (pop 40, p_m 0.6): 15/15 hits at mean 384 evals — **still not faster than random**. Best-found quality: random matches/beats PRISM at every budget ≥ 50.
- **Meaning: the searcher has no advantage on locally-random landscapes.** Landscape locality — not the algorithm — governs when evolutionary search pays. Decisions D12, D13. This falsification redirected the whole program toward the protocol.
- **Artifacts:** `experiments/iteration-07/`.

4.8 Experiment J — The pre-flight diagnostic (Iteration 8)

- **Problem:** can the outcome of Experiment I be *predicted* before searching?
- **Setup:** rho1 (one-step move autocorrelation, per operator, 4,000 sampled moves) + FDC (fitness–distance correlation, Cayley distance to nearest optimum) on 8 fully enumerated landscapes; ~2 min CPU, zero new training.
- **Mode:** statistics on cached landscapes (no genetic ops).
- **Results:** rho1 recovers the matched operator 3/3 (hamming→swap 0.655, kendall→insert 0.777, adjacency→inversion 0.675). FDC separates all 8 observed search outcomes: –0.83 (fast convergence) … –0.06 (parity n=7: PRISM ≈ random) … +0.78 (deceptive). Neural landscapes: rho1 ≈ 0.00–0.06 — locally near-random, *explaining Experiment I from pre-search statistics alone*.
- **Meaning (D14):** the rho1+FDC pre-flight became mandatory for every new application. Reliability was later measured (It-14): operator pick 94–99% correct at 100 move-pairs; deceptive call 100% stable.
- **Artifacts:** `experiments/iteration-08/locality_study.py`, `results/locality.*`.

4.9 Experiment K — LLM instruction ordering (Iteration 9) — THE FLAGSHIP. Full audit detail.

- **Problem brief:** LLM prompts are typically assembled from instruction modules in an arbitrary order. Does the *order alone* — with content, model, and questions frozen — matter? Nobody had ever mapped a complete ordering landscape.

- **Exact configuration (auditable):**
- **Model:** `gemini-2.5-flash-lite`, temperature 0, maxOutputTokens 800, via the `generateContent` REST API; key from GCP Secret Manager.
- **The six instruction modules, verbatim:**
- `RESTATE: Restate the problem briefly in your own words.`
- `IDENTIFY: List the known quantities and what is being asked.`
- `PLAN: Devise a short step-by-step strategy before calculating.`
- `COMPUTE: Carry out the calculations step by step.`
- `CHECK: Verify the result against the problem statement.`
- `ANSWER: State the final answer on its own line as 'Answer: <number>'.`
- **Prompt template, verbatim** (perm = the permutation being tested):

```
Solve the following math problem. Work through these steps in this
exact order:
```

```
1. <module at perm[0]>
2. <module at perm[1]>
...
6. <module at perm[5]>

Problem: <question text>
```

- **Questions:** a fixed 32-question GSM8K subset (`experiments/iteration-09/data/gsm8k_subset32.json`), identical for every ordering.
- **Grading:** exact match — regex `Answer:\s*\$?(-?[\d,]+(?:\.\d+)?)` (fallback: last number in the reply), compared to gold within 1e-6. Parser frozen across all follow-up experiments.
- **Fitness of an ordering** = fraction of the 32 questions answered correctly.

- **What was done:** staged protocol — a variance gate first (is there any ordering effect worth paying for?), then **exhaustive enumeration of all 720 orderings** (~23,000 cached API calls, ≈$3–5), each call cached by (ordering, question) so the run is resumable and every number is recomputable from the cache.

- **Results:**
- Accuracy ranges **0.063 → 0.969 by ordering alone** (std 0.230). Same model, same 32 questions, same six sentences — only their order differs.
- **Worst three orderings (fitness 0.062):** e.g. `RESTATE → PLAN → CHECK → ANSWER → COMPUTE → IDENTIFY`. **Best three (0.969):** e.g. `CHECK → COMPUTE → IDENTIFY → ANSWER → RESTATE → PLAN`. The audit-friendly pattern: in every worst ordering **ANSWER precedes COMPUTE** (the model is told to state the answer before being told to compute it); in every best ordering COMPUTE precedes ANSWER. This is *precedence structure* — which is exactly why the pre-flight's rho1 picked the precedence-matched **insert** operator.
- **Position effects (marginal averages over the full landscape):** ANSWER placed first averages 0.435, placed last 0.870; COMPUTE placed early 0.908, in fifth position 0.585. (Note for auditors: these are *marginal averages*; the single best orderings put ANSWER 4th with the weak modules trailing — the marginal rule "ANSWER late" and the exact optimum are consistent but not identical.)
- **Pre-flight (registered before the search stage):** rho1 → insert (0.547); FDC –0.346 → "search beats random". Both verified.
- Search race (15 seeds, exact, against the cached landscape): PRISM ~6 distinct evaluations to first optimum vs random ~18. **This "3× faster" was later retracted at 40 seeds (It-14): 9.9 [7,13] vs 10.8 [8,14] — statistically indistinguishable.** Optima are moderately dense (60/720 = 8.3%), which is why speed claims are fragile here.
- **Meaning:** the first complete enumerated prompt-ordering landscape in the literature; a 90-point accuracy swing from reordering six sentences; interpretable, reusable position rules; and a live validation of the pre-flight. The anchor of the LLM paper.
- **Artifacts:** `experiments/iteration-09/llm_chain_experiment.py`, `results/{answer_cache.csv, llm_landscape.csv, llm_position_effects.csv, llm_analysis.txt, stage1_gate.txt}`.

4.10 Experiment L — Aging + surrogate from the literature (Iteration 10)

- **Setup:** aging evolution (Real et al. 2019) and a BANANAS-style positional one-hot ridge surrogate; 4 methods × 3 cached landscapes × 15 seeds; 40-seed verification on parity n=7; cap 500 distinct evals.
- **Results:** aging lifts parity hit rate 48%→78% at equal budget but lands exactly at random's 75% (as FDC≈0 predicts). Elitist fastest on structured landscapes (18.5 vs 28.7 evals, XOR n=6). Surrogate wins only where encoding matches (LLM landscape: 5.8 evals).
- **Meaning (D15):** pre-flight also selects the replacement policy; aging = safe no-pre-flight default.
- **Artifacts:** `experiments/iteration-10/`.

4.11 Experiment M — n=8 LLM instance, beyond enumeration (Iteration 11)

- **Setup: 8 modules** — the six above plus, verbatim: `ESTIMATE: Make a rough order-of-magnitude estimate of the answer.` and `SIMPLIFY: Note any way to simplify the problem before solving.` → 40,320 orderings; 20 GSM8K questions; same model/template/parser; hard budget cap in code; actual spend $5.56 (23,805 calls).
- **What was done:** sampled pre-flight on 222 orderings (registered forecast), then search stage (4 seeds/method).
- **Results:** ordering swings accuracy **0.10** → **1.00** (std 0.240 — larger than n=6). Pre-flight: rho1 → insert 0.75 (precedence holds at n=8); approx-FDC –0.095 → "random likely competitive". Search: PRISM = random = 1.0000 at every budget — the forecast exactly right (perfect orderings are dense at 20-question resolution).
- **Meaning (D16):** establish fitness resolution before search budget.
- **Artifacts:** `experiments/iteration-11/harder_llm_experiment.py`, `results/`.

4.12 Experiment N — Precedence surrogate (Iteration 12) — first method to beat elitist PRISM

- **Setup:** 6 methods × 4 landscapes incl. kendall n=7 (ONE optimum in 5,040 — a needle). Surrogate = ridge regression on binary "i-before-j" precedence features, proposing the next ordering to evaluate.
- **Results:** surrogate finds the needle in **13.5 mean evals vs elitist 55.7 vs random 0/15** (disjoint CIs at 40 seeds). Elitist+restart hybrid: decent everywhere, best nowhere.
- **Meaning (D17):** surrogate power = encoding–landscape match, the exact parallel of operator matching. The program published a method beating its own namesake.
- **Artifacts:** `experiments/iteration-12/`.

4.13 Iteration 14 — 40-seed statistics pass + claims audit (rigor, no new experiment)

- 40-seed Wilson/bootstrap CIs on every headline; 200 resampled pre-flights → operator pick 94–99% correct, deceptive call 100% stable. **Retraction:** K's "3× faster" (above). Kendall surrogate strengthened. Output: `docs/ICML_READINESS_AUDIT.md` (verdict: GECCO/TEVC-ready; LLM landscape = top-tier asset). Decision D18.

4.14 Iteration 15 — Aging-mode theory (Appendix B)

- One-step reachability of any ordering (prob ≥ 1/(2n(n–1)n!)); aging = almost-sure *record* convergence with explicit geometric bound (parity n=7 bound 30,240 vs observed 222.5); population absorption provably fails under aging; the elitist hybrid preserves the original theorem. Numerically verified (2M draws: 2.21e-4 vs bound 2.08e-4). `experiments/iteration-15/results/theory_check.txt`.

4.15 Experiments O & P — Transfer and SciML (Iteration 16)

- **O1 (cross-task):** XOR-v4 vs parity-v4 orderings, 120 shared — transfer r = –0.004, exactly zero. Good orderings are task-specific on the neural benchmarks.
- **O2/O3 (cross-size, LLM n=6→n=8):** position effects from the n=6 landscape predict n=8 fitness at **Spearman 0.665 [0.582, 0.725]**; picking the n=8 top-10% by the n=6-derived score yields accuracy **0.900 [0.873, 0.925] vs random 0.725** — a +17.5-point warm start with zero target evaluations.
- **P (SciML — first of its kind):** a SINDy dynamics-discovery pipeline whose six preprocessing stages (CLIP, SUBSAMPLE, SMOOTH, TRIM, MEDIAN, DIFF) were permuted; all 720 orderings enumerated, deterministic, $0. Equation-recovery error swings **0.1%** ↔ **23.2%** by ordering alone; best ordering CLIP→SUBSAMPLE→SMOOTH→TRIM→MEDIAN→DIFF. Pre-flight FDC +0.109 (borderline) → cautious call; 40 seeds: random 40/40 vs PRISM 38/40 — directionally correct → D19 borderline band.
- **Artifacts:** `experiments/iteration-16/`.

4.16 Experiment R — Transfer-guided evaluation policy (Iteration 18)

- Cached-data-only: the n=6 landscape guides which n=8 orderings to evaluate. Guided evaluation reaches a perfect n=8 ordering by budget 5 (random-without-replacement: p = 0.248). Supporting evidence, not headline — perfect n=8 orderings aren't sparse (12/222) at 20-question resolution. Benchmark #2 exists to fix exactly this.
- **Artifacts:** `experiments/iteration-18/results/transfer_guided_selection.txt`.

4.17 Experiments Q & Q2 — Cross-model / cross-family transfer (Iterations 17 & 19)

- **Problem brief:** everything above used one model. Do ordering effects — the landscape, its structure, its extremes — transfer to *different, smaller* models? This is the second-model replication the audit demanded and the live test of the D20 strategy.
- **Setup (identical for all three targets):** source = the complete Experiment K landscape. Targets scored on **120 orderings** (100 random, seed 17 + source top-15 + source bottom-5) × the same 32 questions = 3,840 cells each, cached, resumable, temperature 0, same prompt template and parser. Targets:
- **Gemma** `gemma-4-26b-a4b-it` (26B MoE, 4B active), Google API — $0 billed (4,101 calls; worst-case-if-billed $1.37);
- **Qwen** `qwen/qwen3-30b-a3b-instruct-2507` (30B MoE, **3B active**; the planned qwen3-4b is not served on OpenRouter), via OpenRouter — $0.45 (3,954 calls);
- **Llama** `meta-llama/llama-3.2-3b-instruct` (3B dense), via OpenRouter — $0.21 (3,900 calls). OpenRouter legs ran under a two-meter $9 hard cost cap.
- **Results (bootstrap CIs, 10k resamples):**

Metric	Gemma	Qwen3-30B-A3B	Llama-3.2-3B
Target mean accuracy	0.442 (mid)	0.918 (**ceiling**)	0.254 (headroom)
Rank transfer (Spearman)	0.158 [–0.018, 0.338]	0.250 [0.098, 0.490]	**0.375 [0.181, 0.554]**
Position-structure r	0.064	0.401	**0.582** (ANSWER row monotone 0.13→0.33)
Top-15 minus random	[+0.008, +0.234]	[+0.013, +0.051]	[+0.003, +0.118]
Random minus bottom-5	[+0.221, +0.372]	[–0.034, +0.096]	[+0.113, +0.200]
Order sensitivity (std)	0.247	0.075	0.153

- **Meaning:** (1) the **top-ordering advantage transfers on all three families** — three independent CIs excluding zero; (2) **bottom-avoidance transfers decisively wherever the target has headroom** and vanishes at ceiling (a strong model solves even badly-ordered prompts); (3) **positional structure transfers with headroom** — the It-17 Gemma-only conclusion ("structure does not transfer") was **corrected** by the Q2 replication; Gemma is the outlier, not the rule; (4) new moderator: **target capability** — order sensitivity collapses at ceiling. Practical claim: screen orderings on a cheap frontier model, deploy to small on-device models, avoid ~30-point pathological orderings at zero target-evaluation cost.
- **Artifacts:** `experiments/iteration-17/` and `experiments/iteration-19-crossfamily/` (caches, token meters, `spend_guard.json`, per-leg reports); live dashboard `deliverables/PRISM_Live_Dashboard.html`.

4.18 Side studies

- **PRISM-RL:** option-route grid (semi-MDP; enumerated best 80.0; inversion hit rate 0.625 vs swap 0.25) and chain curriculum (tabular Q-learning; insert hit rate 1.00). PRISM applies wherever RL exposes a finite ordering surface; only the fitness evaluator is domain-specific. `prism-research/outputs/rl_ordering_*`.
- **Historical synthetic scaling:** hamming/kendall hitting-time exponents 3.14/2.89 (excluding $n \le 5$). Committed outputs; rerun still open.
- **ANASOD–PRISM boundary diagnostic (NAS-Bench groundwork, parallel track):** exhaustive partition of NATS-Bench's 15,625 architectures into 210 operation distributions. Operation *counts* explain η^2 = 0.461/0.556/0.612 of accuracy variance (CIFAR-10/-100/ImageNet16); the residual placement structure is real and dataset-conditional (slice ranges up to 4.96 points; exact FDC –0.412/–0.273/–0.721). Compact-table backend; budget-matched search confirmation pending. `experiments/iteration-19/anasod_boundary.py`.

5. Was it tested on other models? (summary table)

Model	Role	Where
Gemini 2.5 Flash-Lite	source landscapes: full n=6 enumeration; n=8 sampled	Exps. K, M
Gemma 26B-MoE / 4B-active	transfer target #1	Exp. Q
Qwen3-30B-A3B-instruct (3B active)	transfer target #2	Exp. Q2

Model	Role	Where
Llama-3.2-3B-instruct	transfer target #3	Exp. Q2

Four models, three families (Google/Alibaba/Meta), two API providers. Not yet tested: OpenAI/Anthropic-family targets, dense 7–8B class, and any model on *hard* questions — the last is exactly benchmark #2.

6. Corrections record (what an auditor should find before the claims)

- Historical codebase described in the handover **did not exist** — rebuilt (It-2).
- H3 falsified: deeper stacks → zero variance, not more (Exp. E).
- H10 falsified: searcher ≈ random on locally-random neural landscapes (Exp. I).
- "PRISM 3× faster than random" (LLM) **retracted** at 40 seeds (It-14, D18).
- Historical convergence-generation claims: artifacts of noisy fitness (Exp. A).
- It-17 "structure does not transfer cross-model" **corrected** by Q2 replication.

7. Tool calling and agentic use — can this protocol excel there?

Short answer: yes, it is arguably the best-matched unexplored surface, and it is now an explicit next-phase candidate. Nothing agentic has been *run* yet — this section specifies the opportunity honestly.

Why the mapping is natural. An agent is a fixed set of components whose order is a free variable at several levels, all of them permutations of a fixed module set — exactly PRISM's object:

- **System-prompt module order** — the direct generalization of Experiment K: agent prompts contain role, constraints, tool-use rules, output format, safety rules, examples. The K/Q/Q2 results (90-point swings; format-instruction position decisive; pathological orders transfer) make large agentic order effects highly plausible.
- **Tool-description order in context** — which tool definitions come first plausibly biases tool selection (analogous to documented multiple-choice position bias).
- **Pipeline stage order** — plan → retrieve → solve → verify → answer as permutable modules; the SINDy result (P) already proves pipeline-order landscapes exist outside LLMs; opportunity R5 ("cost-aware ordering of LLM planners, solvers, verifiers", score 4.7) is precisely this.
- **Skill/curriculum order** for agent training — the PRISM-RL pilot already showed curriculum-ordering landscapes with a perfect insert hit rate.

Why the protocol (not just the searcher) is the right tool there: agentic evaluations are expensive and noisy — exactly the regime where a $1 pre-flight that says "search / don't search / use the surrogate" pays for itself; and the Q2 transfer result suggests orderings could be screened on a cheap model before deployment on the expensive agent stack.

Benchmarks (from the ranked top-100 list; scores are the program's own relevance/feasibility/impact ratings):

Benchmark	What would be permuted	Fit
ToolBench (4.7)	tool-description order; tool-call pipeline order	direct; success-rate metric
API-Bank (4.7)	tool docs + reasoning-module order in dialogue	direct

Benchmark	What would be permuted	Fit
StableToolBench (4.7)	as ToolBench with a stabilized evaluator (less fitness noise — better for pre-flight)	best first target
m&m's (4.7)	multi-modal multi-step tool-plan stage order	pipeline-level
PlanBench (4.7)	plan-step and prompt-module order, verifiable plans	cheap, exact-match
ALFWorld / WebShop (4.0)	action-macro and instruction order	second wave
GAIA (3.7, ambitious)	full-stack agent config order	only after the above

Proposed first experiment (Benchmark #6 candidate, after #2): StableToolBench or PlanBench subset; 6–8 agent prompt modules (role, tool rules, format, plan, act, verify); pre-flight first (registered); 120–250 orderings × ~50 tasks on an SLM; measure success rate, tool-call validity, token cost per task; race guided vs random; test whether K's position rules transfer into the agentic prompt. Est. $5–15 at SLM rates under the existing cost guard. **Known risks, stated in advance:** adaptive policies may beat any fixed order (the agent loop can reorder itself); invalid tool sequences need repair/constraint handling; agentic fitness is noisier — the variance gate (D16) must pass before enumeration-scale spend.

8. The biggest takeaway, and what should happen next

The takeaway in three sentences. Component ordering is a large, structured, *and measurable* performance factor — a 90-point accuracy swing in LLM prompts, a factor-230 error swing in a scientific pipeline — and whether any search will exploit it better than random guessing can be predicted for about a dollar of evaluations before committing budget, with 94–99% measured reliability. Ordering knowledge measured once on a cheap model transfers: across sizes, and across model families, moderated by how much headroom the target has. The searcher is a fine tool; **the protocol — measure, forecast, select, execute, verify — is the contribution.**

What should happen next, in order, and why:

- **Benchmark #2 (Experiment S, ~$5–7):** the hard-question, sparse-optimum landscape. It closes the program's two remaining weaknesses at once (optimum density; the Qwen ceiling caveat) and can turn the transfer-guided policy into the headline. Fully specified with registered hypotheses H19–H23 in the consolidated status document §11.
- **Prompt-optimizer baselines (~$5–10, required by D21):** OPRO/APE, MIPROv2, GEPA, GREATER; complementarity experiment (freeze optimized content, permute its order). Without this, Paper B has a hole every reviewer will find.
- **The free items in parallel:** SciML suite (benchmark #4, $0 — turns Exp. P into a releasable benchmark suite); NAS-Bench search-stage confirmation on the ANASOD groundwork ($0); rho1-vs-alternatives ablation ($0, approved).
- **Then the agentic pilot (§7)** — the natural expansion, done pre-flight-first so a negative is cheap and a positive opens the largest application surface.
- **Paper B assembly and submission** (LLM venue), Paper A to GECCO/TEVC — the competitive window is open but closing (order-sensitivity literature is accelerating; checked 2026-07-09).

Companion documents: `docs/RESEARCH_STATUS_COMPREHENSIVE_2026-07-12.md` (the consolidated status, incl. benchmark #2 spec §11), `docs/EXPERIMENT_LOG.md` (primary record), `deliverables/audio/lesson-7-complete-status.mp3` (40-min narration). PDF of this report: `docs/EXPERIMENTS_AUDIT_REPORT_2026-07-12.pdf`.